

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Introducción a la Computación 22.06

Arreglos y Estructuras de Datos

Trabajo de Laboratorio N ° 3

Curso 2020A – 1er Cuatrimestre

GRUPO N°:4		INTEGRANTES
Legajo N°	Nombre	
62490	Marcos Dario Alegre	
62012	Santiago Feldman	
62447	Anna Candela Gioia Perez	
61758	Juan Sebastián Helou	

	Fecha	Docente
Realizado	11/10/2020	
Presentado	14/10/2020	
Aprobado		

Introducción a la Computación

Trabajo práctico N° 3

Arreglos y Estructuras

Ejercicio N° 1

Crear un arreglo de 6 bytes no signados con los siguientes valores iniciales:
{12, 15, 94, 20, 15, 45}.

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           12/Oct/20  13:19:28  #
#
#   Source   =   tp3ej1a.msa                      #
#   List     =   tp3ej1a.lst                      #
#   Object   =   tp3ej1a.r07                     #
#   Options  =                                     #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                p68h11                ;declara el microprocesador
2  *****
3  *                                ÁREA DE ARREGLO                                *
4  *   Se crea un arreglo de 6 elementos de un byte cada uno                *
5  *****
6  2000      RAM      equ      $2000
7  0006      N        equ      6              ;declara la cantidad de elementos del arreglo
8  0001      EL_SIZE  equ      1              ;declara la cantidad de bytes de los elementos del arreglo
9
10
11
12  *****
13  *                                ÁREA DE DATOS                                *
14  *   Lista de los elementos que pertenecen al arreglo                    *
15  *****
16  2000      org      RAM
17  2000      arreglo rmb  EL_SIZE * N      ;reserva la posiciones de memorias del arreglo
18  2000      org      arreglo
19  2000 0C0F5E14      FCB      12,15,94,20,15,45      ;define los elementos del arreglo
    2004 0F2D
20
21  2006                END

Errors:  None      #####
Bytes:   6         # tp3ej1a #
CRC:    5B16      #####
```

- Crear un arreglo de 6 enteros no signados (2 bytes c/u). Los valores iniciales deben ser: {1200, 1235, 44694, 11920, 50915, 45}.

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           12/Oct/20  13:17:33  #
#
#   Source   =   tp3ej1b.msa                               #
#   List     =   tp3ej1b.lst                               #
#   Object   =   tp3ej1b.r07                               #
#   Options  =                                           #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                p68h11                ;declara el microprocesador
2
3          *****
4          *                      ÁREA DE ARREGLO                      *
5          *   Se crea un arreglo de 6 elementos de dos byte cada uno   *
6          *****
7  2000      RAM      equ      $2000
8  0006      N        equ      6          ;declara la cantidad de elementos del arreglo
9  0002      EL_SIZE  equ      2          ;declara la cantidad de bytes de los elementos del arreglo
10
11
12
13
14          *****
15          *                      ÁREA DE DATOS                      *
16          *   Lista de los elementos que pertenecen al arreglo        *
17          *****
18  2000                org      RAM
19  2000      arreglo  rmb      EL_SIZE * N          ;reserva las posiciones de memoria del arreglo
20  2000                org      arreglo
21  2000  04B004D3      FDB      1200,1235,44694,11920,50915,45 ;define los elementos del arreglo
    2004 AE962E90
    2008 C6E3002D
22  200C                END

Errors:  None          #####
Bytes:   12            # tp3ej1b #
CRC:     0471          #####
```

Ejercicio N° 2

Escribir una rutina que encuentre el mayor de los elementos del array del ejercicio anterior formado por elementos de 1 byte. Indicar los cambios que se deberían hacerse en el código para que la rutina escrita se pueda utilizar para encontrar el elemento

```
#####
#
#   Micro Series 6801 Assembler V2.00/DOS           12/Oct/20  13:18:14  #
#
#   Source   =   tp3ej2f1.msa                      #
#   List     =   tp3ej2f1.lst                      #
#   Object   =   tp3ej2f1.r07                     #
#   Options  =                                     #
#
#                                           (c) Copyright IAR Systems 1990 #
#####

1  0000                p68h11                ;declara el micropocesor
2
3
4          *****
5          *                ÁREA DE ARREGLO                *
6          *   Se crea un arreglo de 6 elementos de un byte cada uno   *
7          *****
8  2000      RAM      equ      $2000
9  0006      N        equ      6          ;declara la cantidad de elementos del arreglo
10 0001      EL_SIZE equ      1          ;declara la cantidad de bytes de los elementos del arreglo
11 3100      TERM     equ      $3100      ;posición de memoria donde se anota el elemento mayor
12
13          *****
14          *                MAIN PROGRAM                *
15          *   Compara los elementos del arreglo y el mayor de todos   *
16          *   lo guarda en la posición de memoria deseada            *
17          *****
18 3000                org      $3000
19 3000 8600          ldaa      #0
20 3002 C601          ldab      #EL_SIZE
21 3004 CE2000        ldx       #arreglo
22 3007 18CE0006      ldy       #N          ;inicia contador de elementos a comparar
23 300B 3D           mul
24 300C 3A           abx
25 300D A600      menor  ldaa      0,x      ;toma un elemento del arreglo
26 300F 08      mayor  inx          ;ubica el puntero al elemnto siguiente
27 3010 1809      dey          ;decrementa el contador
28 3012 2706      beq      anoto ;si el contador es cero termina de comparar
29 3014 A100      cmpa      0,x      ;compara el elemento con el que le sigue
30 3016 22F7      bhi      mayor ;si es mayor el primero, lo compara con otro
31 3018 20F3      bra      menor ;elemento. Si es menor el primero, compara el que
32 301A B73100      anoto  staa      TERM ;le sigue con otro elemento.Anota el elemento mayor
33 301D 7E301D      fin     jmp      fin ;finaliza el programa
34
35          *****
36          *                ÁREA DE DATOS                *
37          *   Lista de los elementos que pertenecen al arreglo        *
38          *****
39 2000                org      RAM
40 2000      arreglo rmb      EL_SIZE * N      ;reserva las posiciones necesarias para el arreglo
41
```

```

42 2000          org      arreglo
43 2000 0C0F5E14 fcb      12,15,94,20,15,45 ;define los elementos del arreglo
    2004 0F2D
44
45 2006          END

```

```

Errors:  None      #####
Bytes:   38        # tp3ej2f1 #
CRC:     58A3      #####

```

Los cambios que creímos necesarios para que el programa encuentre dicho elemento fue usar el acumulador D ya que ahora los elementos son de 2 bytes, además de incrementarlo de a dos veces y por último cambiar el EL_SIZE pasa a ser 2 (en vez de 1 como el anterior).

A continuación, reescribimos el código aplicando dichos cambios:

```

#####
#
#   Micro Series 6801 Assembler V2.00/DOS           12/Oct/20  13:21:40  #
#
#   Source   =   tp3ej2f2.msa                      #
#   List     =   tp3ej2f2.lst                      #
#   Object   =   tp3ej2f2.r07                      #
#   Options  =                                     #
#
#                                                    (c) Copyright IAR Systems 1990 #
#####

```

```

1 0000          p68h11          ;declara el micropocesor
2
3              *****
4              *                ÁREA DE ARREGLO                *
5              *                Se crea un arreglo de 6 elementos de dos byte cada uno *
6              *****
7 2000          RAM      equ     $2000
8 0006          N        equ     6          ;declara la cantidad de elementos del arreglo
9 0002          EL_SIZE  equ     2          ;declara la cantidad de bytes de los elementos del arreglo
10 3100         TERM     equ     $3100 ;posición de memoria donde se anota el elemento mayor
11
12              *****
13              *                MAIN PROGRAM                *
14              *                Compara los elementos del arreglo y el mayor de todos *
15              *                lo guarda en la posición de memoria deseada          *
16              *****
17
18 3000          org      $3000

19 3000 8600          ldaa     #0          ;define el indice que empieza a comparar
20 3002 C602          ldab     #EL_SIZE
21 3004 CE2000         ldx      #arreglo
22 3007 18CE0006       ldy      #N          ;inicia contador de elementos a comparar
23 300B 3D            mul
24 300C 3A            abx
25 300D EC00          menor  ldd     0,x          ;toma un elemento del arreglo

```

```

26 300F 08      mayor  inx      ;ubica el puntero en el siguiente elemento
27 3010 08      inx
28 3011 1809     dey      ;decrementa el contador de elementos
29 3013 2707     beq      anoto   ;si el contador es cero termina de comparar
30 3015 1AA300   cpd      0,x     ;compara el elemento con el que le sigue
31 3018 22F5     bhi      mayor   ;si es mayor el primero, lo compara con otro
elemento
32 301A 20F1     bra      menor   ;si es menor el primer, compara el siguiente con
otro elemento
33 301C FD3100   anoto  std      TERM ;anota el mayor elemento
34 301F 7E301F   fin      jmp      fin ;finaliza el programa
35
36
37 *****
38 *                      ÁREA DE DATOS                      *
39 *          Lista de los elementos que pertenecen al arreglo          *
40 *****
41 2000          org      RAM
42 2000          arreglo rmb  EL_SIZE * N ;reserva las posiciones necesarias para el arreglo
43
44 2000          org      arreglo
45 2000 04B004D3 fdb      1200,1235,44694,11920,50915,45 ;define los elementos del arreglo
46 2000          ;a comparar
2004 AE962E90
2008 C6E3002D
47 200C          END

Errors:  None      #####
Bytes:   46        # tp3ej2f2 #
CRC:    4BFB      #####

```

Ejercicio N° 3

Crear un arreglo bidimensional de 4 x 3 celdas, las mismas contienen números no signados de 16 bits (sólo definirlo, colocar valores en el mismo no es necesario).

Escribir la subrutina **get_cell** que retorne el valor cualquiera de una celda de la matriz de 4x3. La subrutina deberá devolver el valor en el registro D, recibirá por stack la dirección de comienzo de la matriz, el número de fila y columna. La primera fila/columna del arreglo se indica con el número cero, mientras que la última con el 3/2. La función deberá invocar a la subrutina **chk_status** que verificará que los valores de fila y columna sean válidos. La forma de recepción y envío de parámetros a **chk_status** la definirá cada grupo según crea más conveniente. Se exige documentación clara al respecto. La función **get_cell** devuelve el flag de carry en uno si ocurrió un error y en cero si se pudo obtener el valor deseado.

Trabajar con constantes simbólicas en el desarrollo de las funciones.

```

#####
#
#   Micro Series 6801 Assembler V2.00/DOS      13/Oct/20  23:44:01  #
#
#   Source   =   tp33.msa                      #
#   List     =   tp33.lst                      #
#   Object   =   tp33.r07                      #
#   Options  =                               #
#
#

```

```
# (c) Copyright IAR Systems 1990 #
#####
1
2 *****
3 * EJERCICIO 3 *
4 * *
5 * El programa retorna el valor de una celda particular *
6 * perteneciente a un arreglo bidimensional de 4x3 celdas. *
7 *****
8
9 0000 p68H11 ;declara el modelo del microprocesador
10 0004 M EQU 4 ;cantidad de filas
11 0003 N EQU 3 ;cantidad de columnas
12 0002 EL_SIZE EQU 2 ;tamaño de los elementos de la matriz
13 2200 matriz EQU $2200
14 0006 AR_SIZE EQU EL_SIZE*N
15 0003 i SET 3 ;fila en la que se ubica una celda particular.
16 0002 j SET 2 ;columna en la que se ubica la celda part.
17
18
19 *****
20 * MAIN PROGRAM *
21 *
22 * Llama a la subrutina "get_cell" que retorna el valor de la *
23 * celda requerida. Realiza el pasaje de parámetros por stack de *
24 * datos requeridos por la subrutina. *
25 *****
26
27 2000 ORG $2000
28 2000 8E2663 lds #stk_ini ;inicia el stack
29 2003 CE2200 ldx #matriz ;carga la matriz en el puntero x
30 2006 8603 ldaa #i ;carga la fila
31 2008 C602 ldab #j ;carga la columna
32 200A 3C pshx ;pasa la posición de memoria donde inicia la
33 200B 36 psha ;matriz al stack ,pasa la fila y columna de
34 200C 37 pshb ; una celda particular de la matriz al stack
35 200D 8604 ldaa #M ;carga la cantidad de filas
36 200F C603 ldab #N ;carga la cantidad de columnas
37 2011 36 psha ;pasa la cantidad de filas y columnas al stack
38 2012 37 pshb
39 2013 8606 ldaa #AR_SIZE
40 2015 C602 ldab #EL_SIZE
41 2017 36 psha ;pasa al stack el producto entre el tamaño de
42 2018 37 pshb ; los elementos de la matriz y su cantidad de
43 2019 4F clra ;columnas.Tamaño de elementos de la matriz a stack
44 201A 36 psha ;deja dos espacios libres en el stack para
45 201B 36 psha ;guardar el espacio col,fil particular
46 201C BD2023 jsr get_cell ;salta a la subrutina "get_cell"
47 201F 32 pula ;guarda el valor de la celda deseada en el
48 2020 33 pulb ; registro D
49 2021 20FE fin bra fin ;finaliza el programa
50
51 *****
52 * SUBROUTINA GET_CELL *
53 *
```

```

54      * Carga el valor de una celda particular de la matriz de 4x3.      *
55      * Antes de retornarlo al programa principal, llama a la          *
56      * subrutina "chk_status", que verifica que tal valor sea válido.*
57      * El pasaje de los parámetros requeridos por tal subrutina se      *
58      * realiza por stack; son los mismos requeridos por la subrutina *
59      * "get_cell".                                                    *
60      *****
61
62  2023      get_cell      ORG      *
63  2023 3C      pshx                      ;hace un backup del registro X
64  2024 30      tsx                      ;crea un frame pointer (FP) en el stack
65  2025 183C     pshy                      ;hace backup del resto de los registros
66  2027 37      pshb
67  2028 36      psha
68  2029 1AEE0C   ldy      12,X             ;carga la posición donde inicia la matriz
69  202C A60B     ldaa     11,X             ;carga la fila de la celda particular (i)
70  202E E607     ldab     7,X             ;carga AR_SIZE
71  2030 3D      mul                      ;i*AR_SIZE
72  2031 183A     aby                      ;matriz+i*AR_SIZE
73  2033 A60A     ldaa     10,X            ;carga la columna de la celda particular
74  2035 E606     ldab     6,X             ;carga EL_SIZE
75  2037 3D      mul                      ;j*EL_SIZE
76  2038 183A     aby                      ;matriz+i*AR_SIZE+j*EL_SIZE luego carga el
77  203A 18EC00   ldd      0,Y             ; contenido de la celda particular,guarda
78  203D E605     ldab     5,X             ; el byte menos significativo y guarda dicho
79  203F A604     ldaa     4,X             ; contenido en la zona de stack Se guarda el byte más
80  2041 BD204A   jsr      chk_status      ; significativo y salta a la subrutina
81  2044 32      pula                      ; "chk_status", restore de los registros
82  2045 33      pulb
83  2046 1838     puly
84  2048 38      pulx
85  2049 39      rts                      ;retorna al programa principal
86
87      *****
88      * SUBROUTINA CHK_STATUS                                          *
89      *                                                                *
90      * Verifica que los valores de fila y columna sean válidos; es   *
91      * decir, que el elemento requerido pertenezca a la matriz.      *
92      * Devuelve el flag de carry en uno si ocurrió un error y en     *
93      * cero si se pudo obtener el valor deseado.                    *
94      *****
95
96  204A      chk_status      ORG      *
97  204A 3C      pshx                      ;hace un backup del registro X
98  204B 30      tsx                      ;crea el FP
99  204C 183C     pshy                      ;backup del resto de los registros
100 204E 37      pshb
101 204F 36      psha
102 2050 A613     ldaa     19,X             ;carga la fila de la celda particular
103 2052 E611     ldab     17,X            ;carga la cantidad de filas de la matriz
104 2054 5A      decb                      ;decrementa el valor de dicha cantidad,para
105 2055 11      cba                      ; compararlo con la enumeración de las filas
106 2056 220B     bhi      mayor           ; Verifica si el valor de la fila corresponde
107 2058 A612     ldaa     18,X             ; orden de la matriz si no corresponde, salta a
108 205A E610     ldab     16,X            ; "mayor".Carga la columna de la celda

```



```

109 205C 5A          decb          ; particular al cargar la cantidad de columnas
110 205D 11          cba            ; de la matriz realiza el mismo procedimiento
111 205E 2203        bhi      mayor ; para las columnas que con las filas
112 2060 0C          clc            ; el resultado es válido y el flag de carry será 0
113 2061 2001        bra      vuelta
114 2063 0D          sec            ; ocurrió un error, el elemento seleccionado no
115 2064 32          pula            ; pertenece a la matriz y el flag de carry será
116 2065 33          pulb            ; 1. Se hace restore de los registros
117 2066 1838        puly
118 2068 38          pulx
119 2069 39          rts            ; retorna a la subrutina "get_cell"
120
121                  *****
122                  *  ÁREA DE ARREGLO                                *
123                  *                                              *
124                  *  Se define un arreglo bidimensional de 4x3 celdas que      *
125                  *  contienen números no signados de 16 bits.                *
126                  *****
127
128 2200              ORG      matriz
129 2200              RMB      EL_SIZE*M*N
130
131                  *****
132                  *  ÁREA DE STACK                                          *
133                  *                                              *
134                  *  Se necesitan 26 posiciones para el pasaje de parámetros a las *
135                  *  subrutinas correspondientes; se deja un margen extra.      *
136                  *****
137
138 2600              ORG      $2600
139 2600              RMB      100
140 2663              EQU      *-1
141
142 2664              END

Errors:  None          #####
Bytes:   106          # tp33 #
CRC:     19F8         #####

```

Ejercicio N° 4

Crear la estructura *biblioteca* la misma esta compuesta por 20 estructuras *libro*. La estructura *libro* se define de la siguiente manera:

Autor del libro [máx 20 caracteres terminado en cero]

N° de inventario [1 byte]

Tema [5 bytes]

Cada uno de estos campos será completado con caracteres ASCII.

```

#####
#                                                    #
#  Micro Series 6801 Assembler V2.00/DOS          14/Oct/20  11:06:22  #
#                                                    #

```

```
# Source = ej4.msa #
# List = ej4.lst #
# Object = ej4.r07 #
# Options = #
#
# (c) Copyright IAR Systems 1990 #
#####

1 0000 p68h11 ;declaro el microprocesador a usar
2
3 *****
4 * ÁREA DE ARRAY: Se crea un arreglo de 20 elementos de 26 bytes *
5 * cada uno *
6 * *
7 *****
8
9
10 0014 N EQU 20 ;se declara el numero de libros
11 0000 org $0
12 0000 biblioteca EQU * ;se creo la estructura biblioteca
13 0000 libro EQU * ;se creo el elemento libro
14 0000 autor rmb 20 ;se reserva 20 bytes para poner el autor
15 0014 ninvent rmb 1 ;se reserva 1 byte para el numero de inventario
16 0015 tema rmb 5 ;se reserva 5 bytes para el tema
17 001A EL_SIZE EQU *-libro ;declara la cantidad de bytes de los elementos
18 001A rmb (-1+N)*EL_SIZE ; del arreglo. Reservo el espacio para la
19
20 2000 org $2000 ; estructura completa
21 2000 7E2000 fin jmp fin ;termina el programa
22
23 2003 END

Errors: None #####
Bytes: 3 # ej4 #
CRC: 58E6 #####
```

Ejercicio N° 5

Escribir la subrutina **get_author**. La misma recibe el número de inventario de un libro y devuelve un puntero al autor del libro y la cantidad de caracteres que contiene el nombre. Se pide que la función reciba por el registro A el número de inventario, devuelva por IX el puntero y por el registro A la cantidad de caracteres del autor. Si el número de inventario no coincide con uno existente en la biblioteca se debe encender el flag de carry.
Se pide modularidad y claridad en la documentación.

```
#####
#
# Micro Series 6801 Assembler V2.00/DOS 14/Oct/20 14:24:08 #
#
# Source = eje5.msa #
# List = eje5.lst #
# Object = eje5.r07 #
# Options = #
#
# (c) Copyright IAR Systems 1990 #
#####
```

```

1  0000                                p68h11                                ;declaro el microprocesador a usar
2
3
4  *****
5  *  ÁREA DE ARRAY: Se crea un arreglo de 20 elementos de 26 bytes *
6  *  cada uno                                                         *
7  *                                                                     *
8  *****
9
10
11 0014      N      EQU      20      ;se declara el numero de libros
12 0000                                org      $0
13 0000      biblioteca  EQU      *      ;se creo la estructura biblioteca
14 0000      libro      EQU      *      ;se creo el elemento libro
15 0000      autor      rmb      20      ;se reserva 20 bytes para poner el autor
16 0014      ninvent    rmb      1      ;se reserva 1 byte para el numero de inventario
17 0015      tema       rmb      5      ;se reserva 5 bytes para el tema
18 001A      EL_SIZE    EQU      *-libro ;declara la cantidad de bytes de los
19 001A      rmb      (-1+N)*EL_SIZE ; elementos del arreglo, reservo el
20                                ; espacio para la estructura completa
21
22 *****
23 *  MAIN PROGRAM : Aquí se va desarrollar el programa principal que*
24 *  llama a ka subrutinas                                           *
25 *****
26
27
28 2000                                org      $2000
29 2000 8E3063      lds      #stackp      ;necesario para el satck y subrutina
30 2003 BD2009      jsr      get_author   ;se realiza un llamado a la subrutina
31
32 2006 7E2006      fin      jmp      fin      ;finaliza el programa
33 *****
34 *  SUBROUTINA GET_AUTHOR : recibe el número de invenatrio por AccA*
35 *  y devuelve un puntero al autor de libro por IX y la cantidad *
36 *  de caracteres del nombre por AccA                               *
37 *****
38 2009      get_author    org      *
39
40 2009 37                                pshb      ;hace backup de los registros
41 200A 183C                                pshy
42
43 200C 8602      ldaa      #inv      ;se recibe el numero de inventario deseado,
44 200E CE0014      ldx      #ninvent ;debe ser definido previamente.Apunta el
45 2011 18CE0014      ldy      #N      ; registro x al primer numero de
46 2015 C61A      ldab      #EL_SIZE   ; inventario. Crea un contador de libros
47                                ; buscados
48 2017 A100      check    cmpa      0,x      ;chequea si coincide el numero de
49 2019 270A      beq      regauthor ; inventario buscado con el leído. Si es
50 201B 1809      dey      ; el mismo pasa a registrar el autor
51 201D 2602      bne      next      ;si no pasa al siguiente libro inventario
52 201F 0D      sec      ; buscado ,si ya paso el ultimo libro y no encontró
53 2020 39      rts      ; lo buscado prende el carry
54 2021 3A      next      abx      ;pasa al siguiente libro

```

```

55 2022 7E2017      jmp      check
56
57 2025 18CE0014    regauthor    ldy      #ninvent-autor    ;se lleva el pointer al principio del
58 2029 09          point        dex              ;nombre del autor
59 202A 1809          dey
60 202C 26FB          bne      point
61
62 202E 3C          pshx              ;se guarda esta ubicación porque el pointer va
63 202F 4F          clra              ; a ser modificado ,crea el contador de caracteres
64
65 2030 E600          authornam     ldab     0,x          ;lee el caracter
66 2032 C100          cmpb     #0          ;se fija que no sea 0
67 2034 2705          beq      namelength    ;si es 0 deja de leer
68 2036 4C          inca              ;incrementa el contador de caracteres
69 2037 08          inx
70 2038 7E2030      jmp      authornam     ;pasa al siguiente caracter
71
72 203B 38          namelength     pulx              ;recupera la direccion de inicio del nombre
73                                     ; del autor
74 203C 1838          puly              ;hace restore de los registros
75 203E 33          pulb
76
77 203F 39          rts              ;vuelve al main program
78
79
80
81 *****
82 *  ÁREA DE VARIABLES: Donde se declaran las variables a usar      *
83 *                                                                    *
84 *****
85
86 0002          inv      EQU      2          ;el número de inventario deseado
87
88
89 *****
90 *  ÁREA DE STACK: se  reserva las posiciones del stack con un      *
91 *    margen extra                                                  *
92 *****
93 3000          org      $3000
94 3000          stack    rmb      100
95 3063          stackp   EQU      *-1
96 3064          END

Errors:  None          #####
Bytes:   64            # eje5 #
CRC:     6B6B          #####

```

Nota: Todos estos problemas deberán ser realizados haciendo uso del compilador y sus directivas. Se evaluarán los criterios de buena programación estipulados por el documento del IOL publicado por la cátedra.

Fecha de entrega: 14 de Septiembre de 2020